

BUNKERWEB WAF

Web Application Firewall Security Validation

Flask Web Application Protection • ModSecurity • Attack Detection • Blocking • Reporting • Ban Enforcement

Test Area	Observed Result
SQL Injection	Detected and blocked by ModSecurity/libinjection
Cross-Site Scripting (XSS)	Detected by XSS rules and blocked
Path Traversal	Detected and blocked with 403 response
Security Reporting	Incidents recorded in BunkerWeb Reports
Enforcement	Service-specific ban shown in BunkerWeb

Scope: Controlled web-security testing of the locally hosted Flask application through the BunkerWeb WAF. The screenshots below are included as evidence of the observed security behavior.

1. Project Overview

This project demonstrates the deployment and validation of BunkerWeb as a Web Application Firewall (WAF) in front of a Flask web application. The objective of the validation phase is to confirm that suspicious HTTP requests are identified by the ModSecurity ruleset, rejected at the WAF layer, recorded in the BunkerWeb security reports, and-where enforcement is triggered-associated with a ban.

1.1 Security Architecture

Client → BunkerWeb WAF → Flask Web Application

Normal requests are intended to pass through the reverse-proxy layer to the backend application. Requests matching malicious patterns are intercepted by BunkerWeb/ModSecurity and returned as a 403 Forbidden response. The backend remains available, which is visible in the BunkerWeb error page where the web server is marked Working.

1.2 Validation Objectives

The testing focused on three representative web-attack categories: SQL Injection, Cross-Site Scripting (XSS), and Path Traversal. For each category, the test input was submitted through the application and the resulting WAF decision was reviewed in the browser and in BunkerWeb Reports. The Bans page was then used to verify enforcement behavior.

1.3 End-to-End Security Workflow

Attack input → HTTP request → BunkerWeb inspection → ModSecurity rule match → WAF block → 403 response → Security Report → Ban/enforcement

2. Security Test Results

Test	WAF Evidence	Outcome
SQL Injection	Rule 942100; libinjection detection; username argument	Blocked
XSS	Rules 941100 and 941110; script-tag vector	Blocked
Path Traversal	Rules 930100 / 930110; traversal sequence	Blocked
Client response	BunkerWeb 403 Forbidden page	Enforced
Ban enforcement	Bans page; reason: modsecurity	Observed

3. Detailed Evidence and Explanation

Each screenshot below is presented with a technical explanation describing what is visible, what the WAF is doing, and why the evidence is important.

Screenshot 1- BunkerWeb 403 Forbidden Response

The browser receives a 403 Forbidden response from the protected service. The BunkerWeb page explicitly shows the client as Forbidden while BunkerWeb and the web server are Working. This confirms that the security layer is actively intercepting the request rather than the backend server being unavailable.

Key evidence: The 403 response is generated while both BunkerWeb and the web server remain operational.

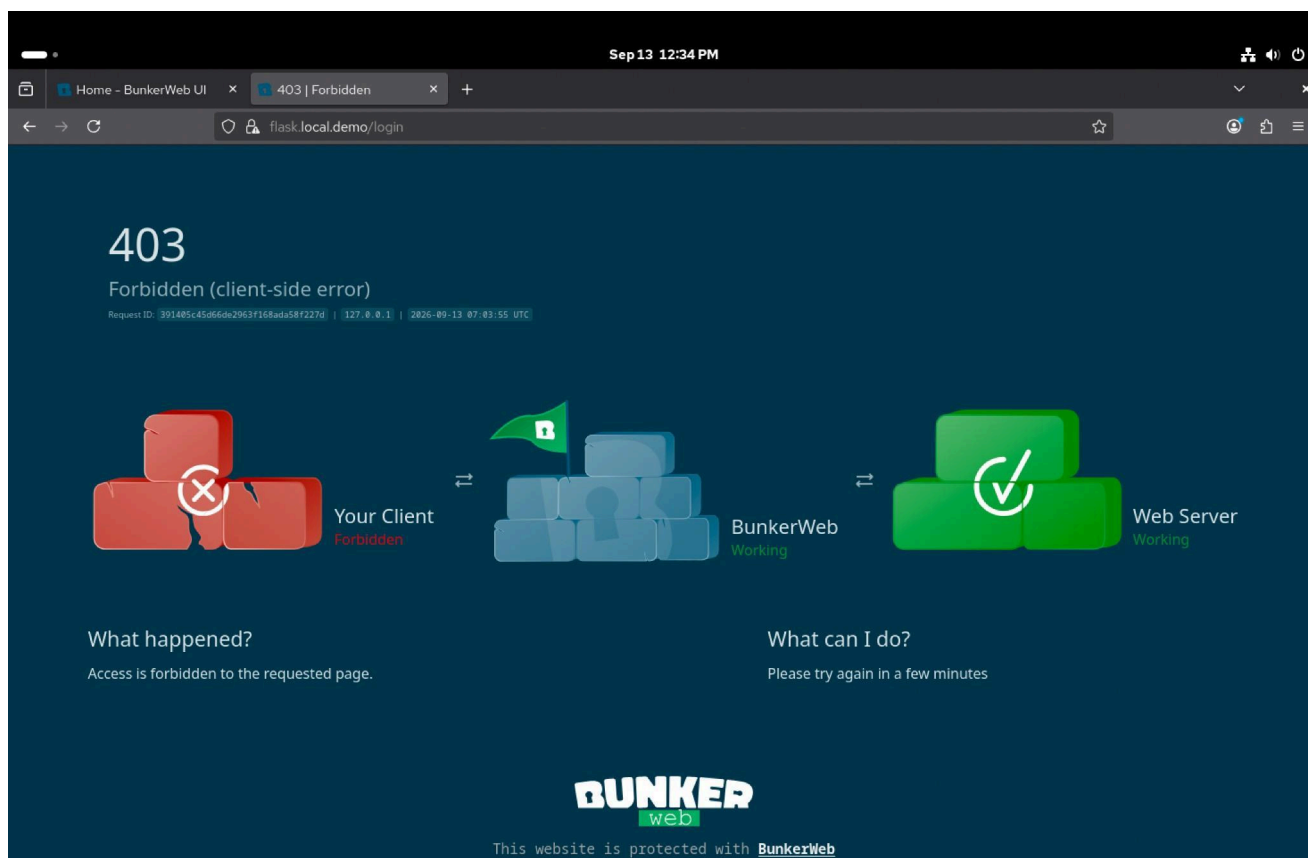


Figure 1. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 2- SQL Injection Detection Report

The BunkerWeb Reports interface records a ModSecurity security incident with Rule ID 942100 and identifies an SQL Injection attack detected via libinjection. The affected variable is ARGS:username and the submitted test value is shown in the report. This provides direct evidence that the WAF identified the malicious input.

Key evidence: Rule 942100 classifies the request as SQL Injection and identifies the affected username argument.

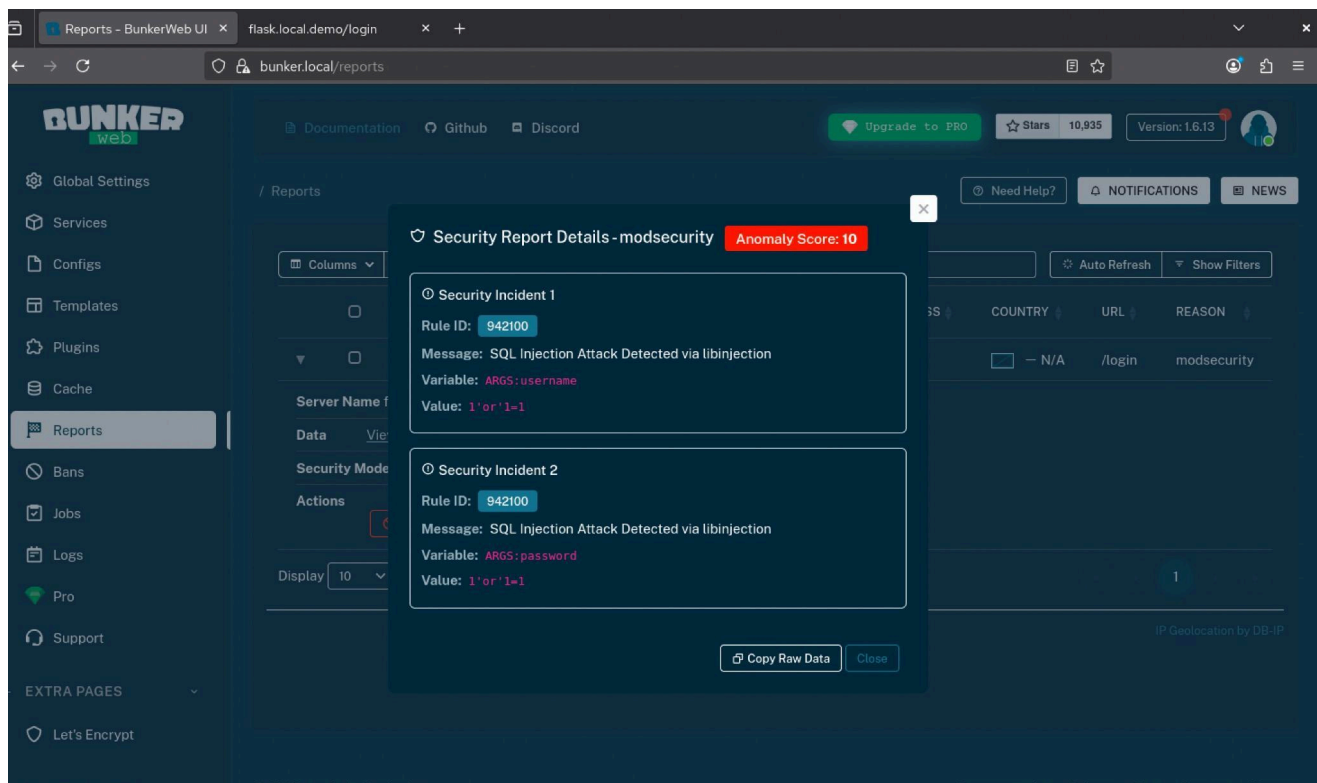


Figure 2. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 3- SQL Injection Test on the Login Form

The login form is used as the controlled test point. A SQL-injection test value is entered in the Username field while the Password field is also supplied with test input. The purpose is to validate whether malicious input reaches the application and whether the WAF can detect it before normal application processing.

Key evidence: The login input is being used as the controlled attack vector for WAF validation.

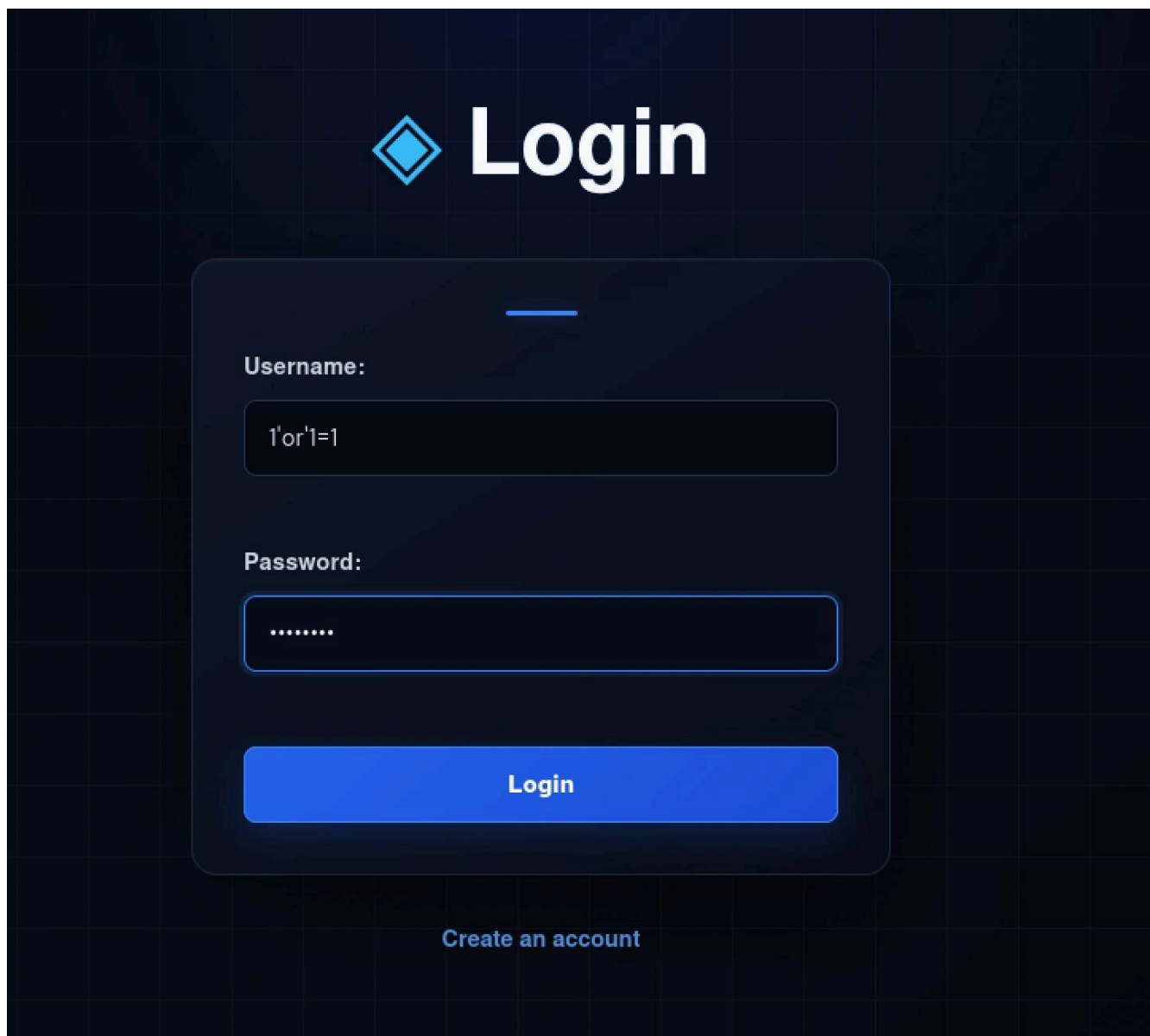


Figure 3. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 4- XSS Payload Test

A script-tag payload is entered into the Username field to test cross-site scripting detection. This is a controlled, non-destructive payload used only to validate the WAF's filtering capability in the lab environment.

Key evidence: The script-tag test is designed to trigger XSS detection without performing a destructive action.

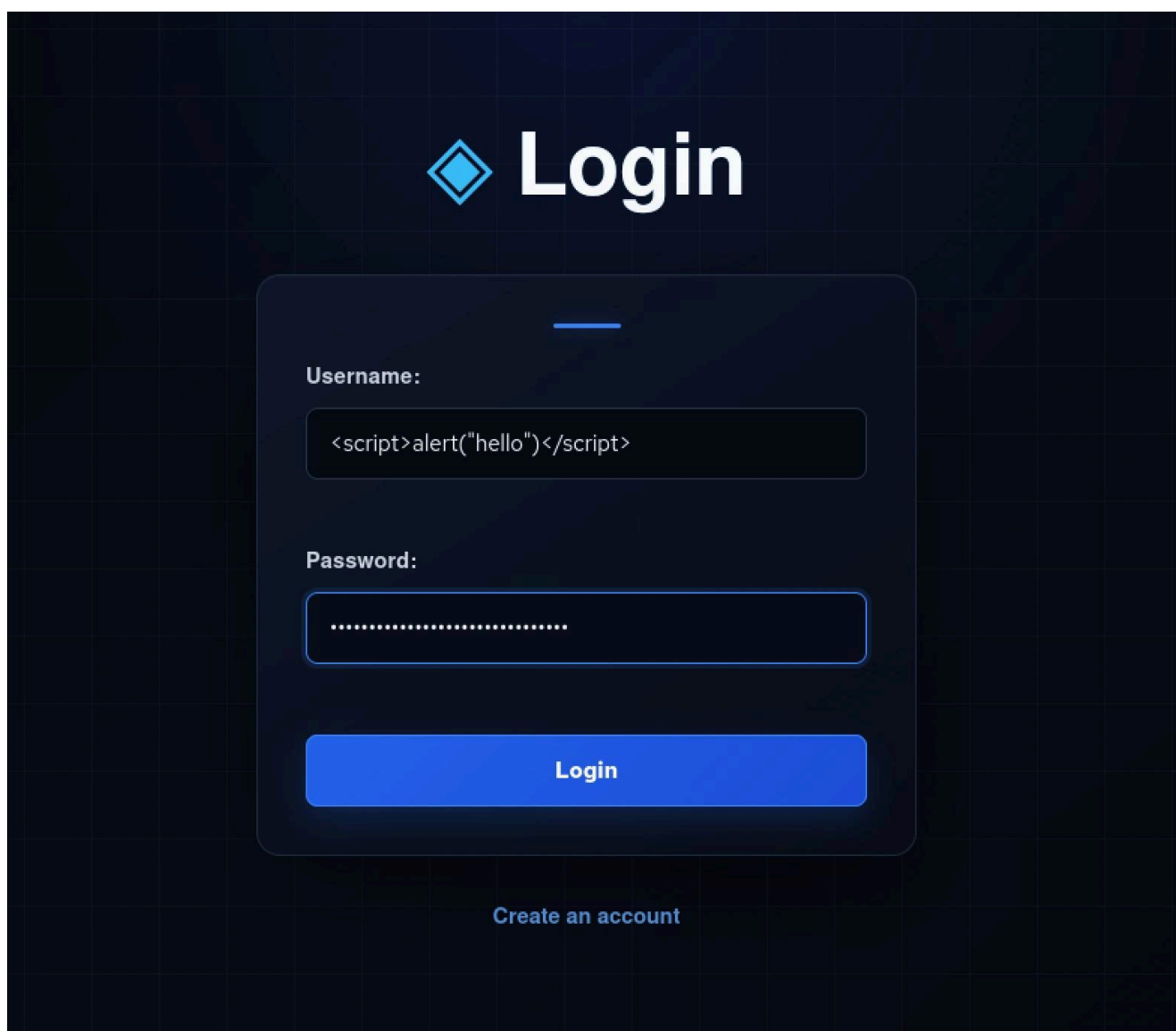


Figure 4. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 5- XSS Detection in BunkerWeb Reports

The security report records XSS-related detections, including Rule ID 941100 (XSS Attack Detected via libinjection) and Rule ID 941110 (XSS Filter- Category 1: Script Tag Vector). The report identifies the affected request argument and shows an anomaly score, demonstrating that multiple ModSecurity rules can contribute to the security decision.

Key evidence: Multiple XSS rules match the same request, producing a higher anomaly score.

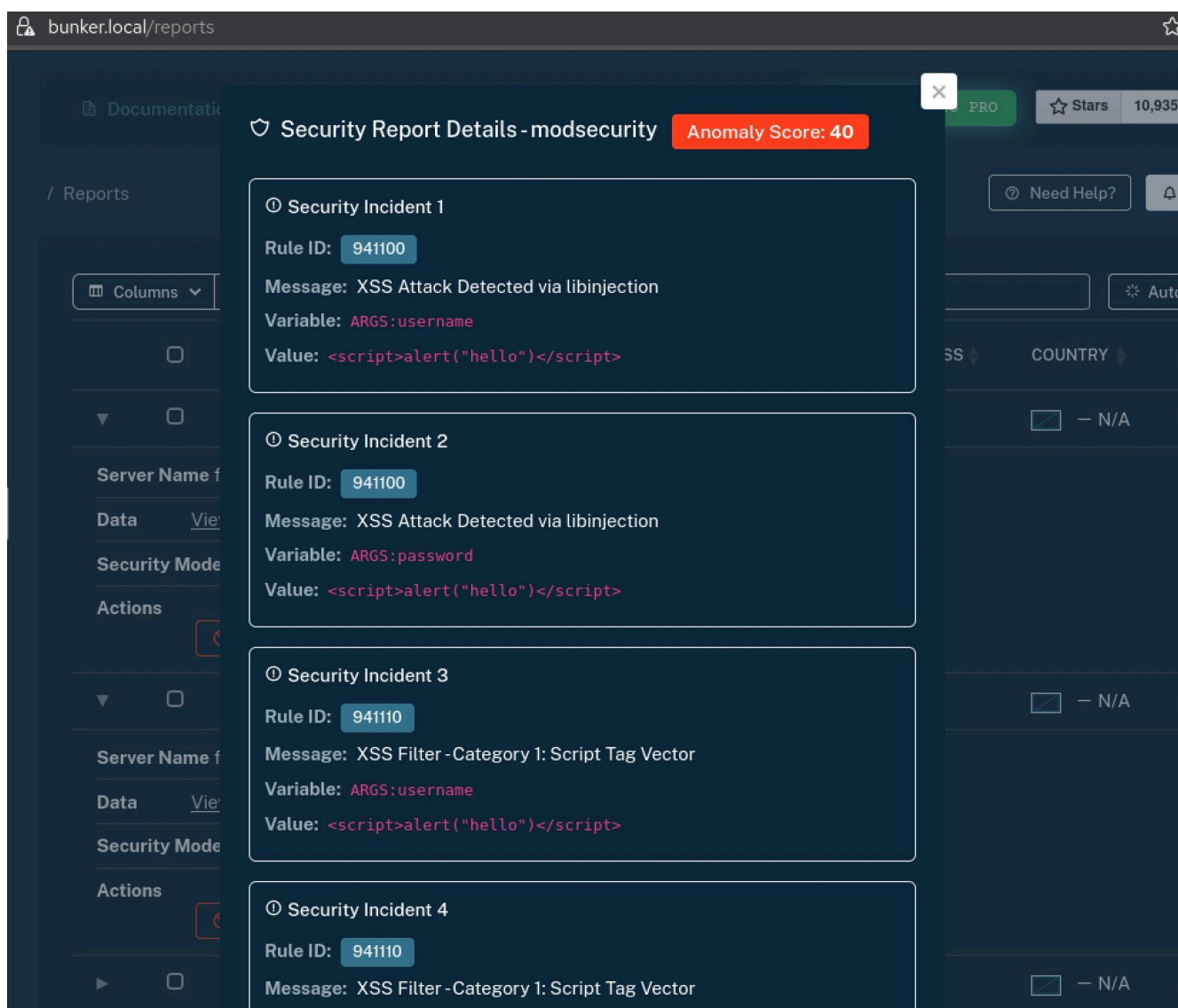


Figure 5. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 6- Path Traversal Detection

The report identifies a Path Traversal attack using Rule ID 930100 and shows the suspicious `REQUEST_URI_RAW/ARGS:file` values. The test attempts directory traversal toward a sensitive operating-system file. The WAF detects the traversal pattern and records the event as a ModSecurity security incident.

Key evidence: Rule 930100 identifies directory traversal in the request URI/argument.

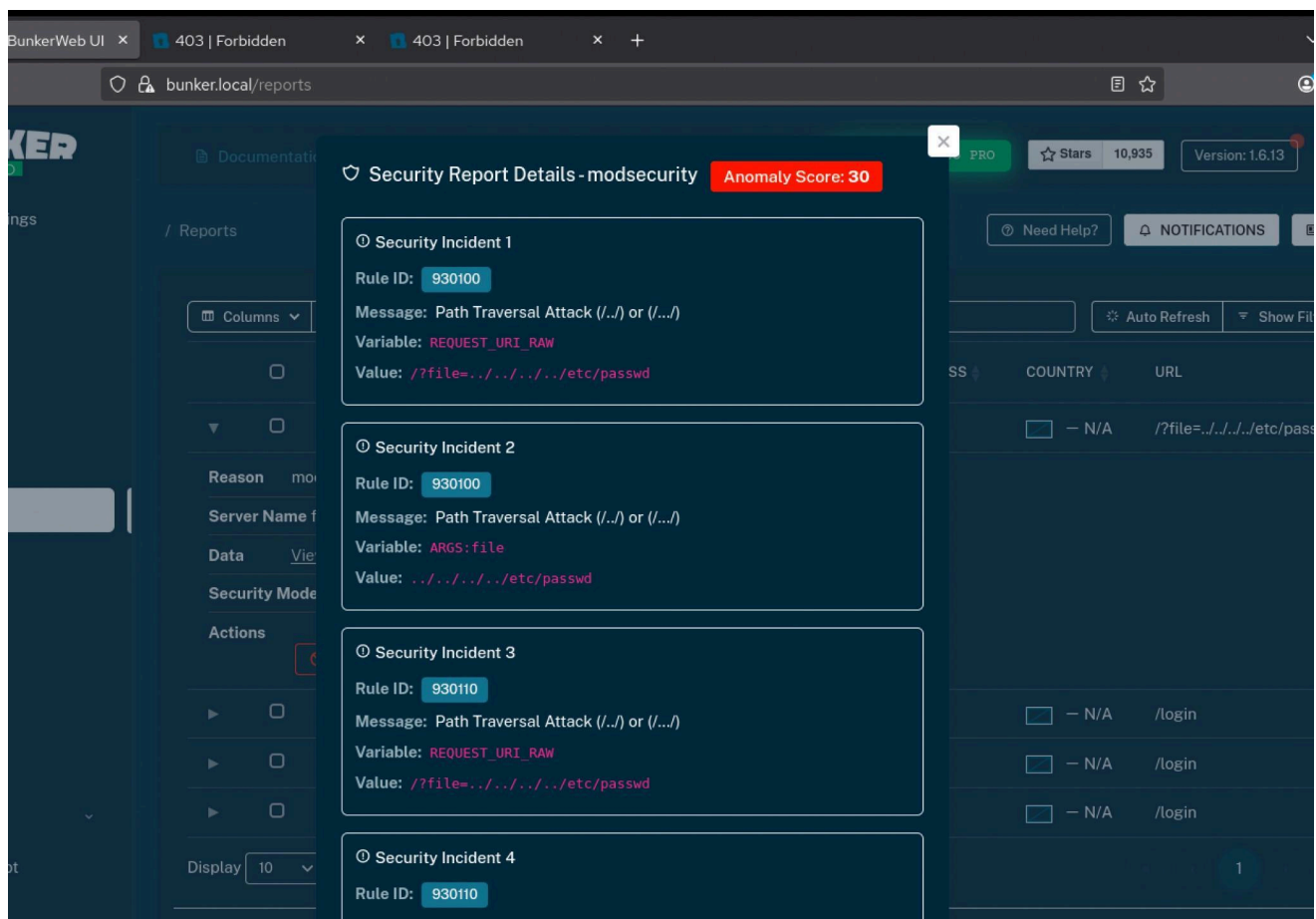


Figure 6. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 7- Path Traversal Blocked with 403

The corresponding browser request is denied with a 403 Forbidden response. BunkerWeb and the web server are still shown as Working, which demonstrates the important distinction between an unavailable backend and a request intentionally rejected by the WAF.

Key evidence: The malicious path request is denied at the WAF layer and does not indicate backend downtime.

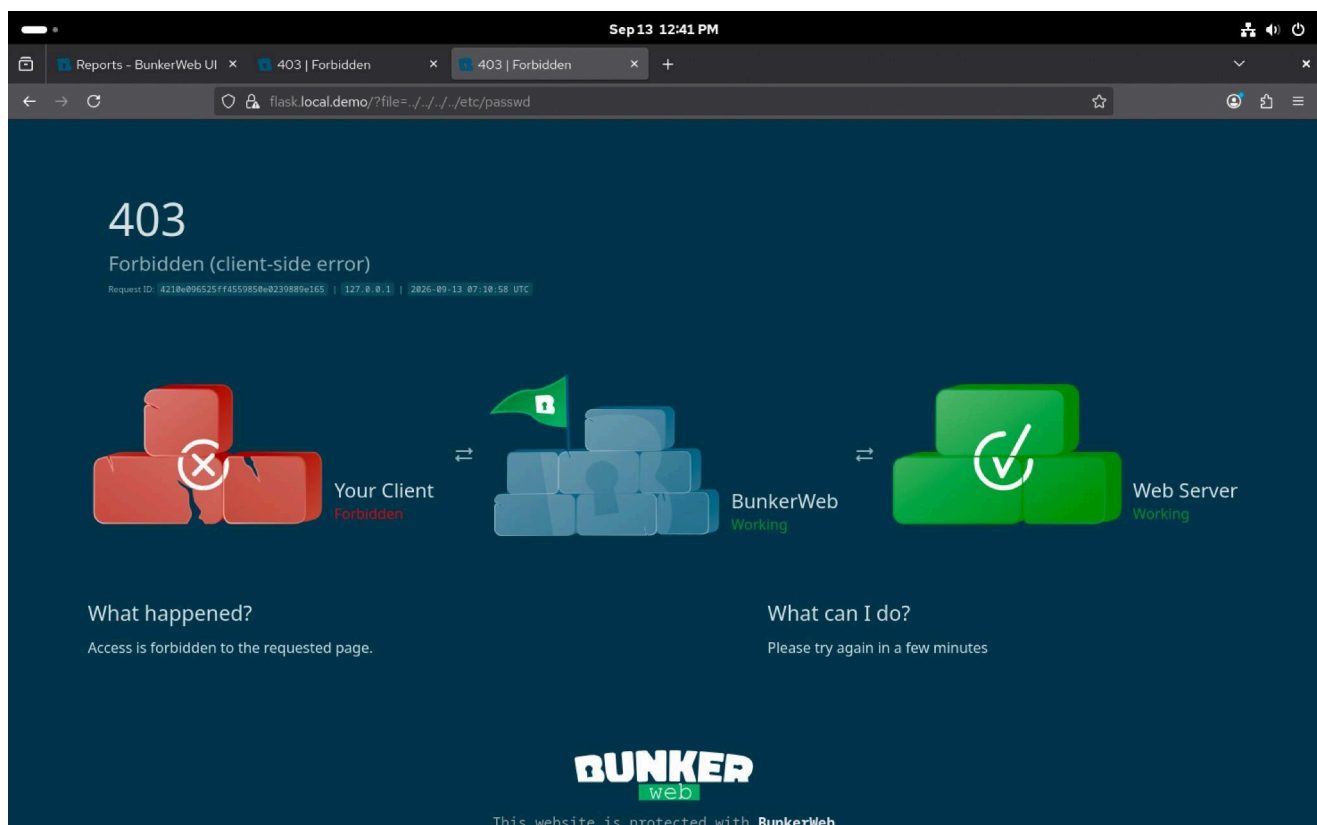


Figure 7. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 8- BunkerWeb IP Ban

The BunkerWeb Bans page shows a service-specific ban associated with the protected Flask service. The reason is listed as modsecurity and the displayed time remaining is approximately 24 hours. This demonstrates the transition from detection and blocking to automated enforcement.

Key evidence: BunkerWeb shows a service-specific ban with ModSecurity as the reason.

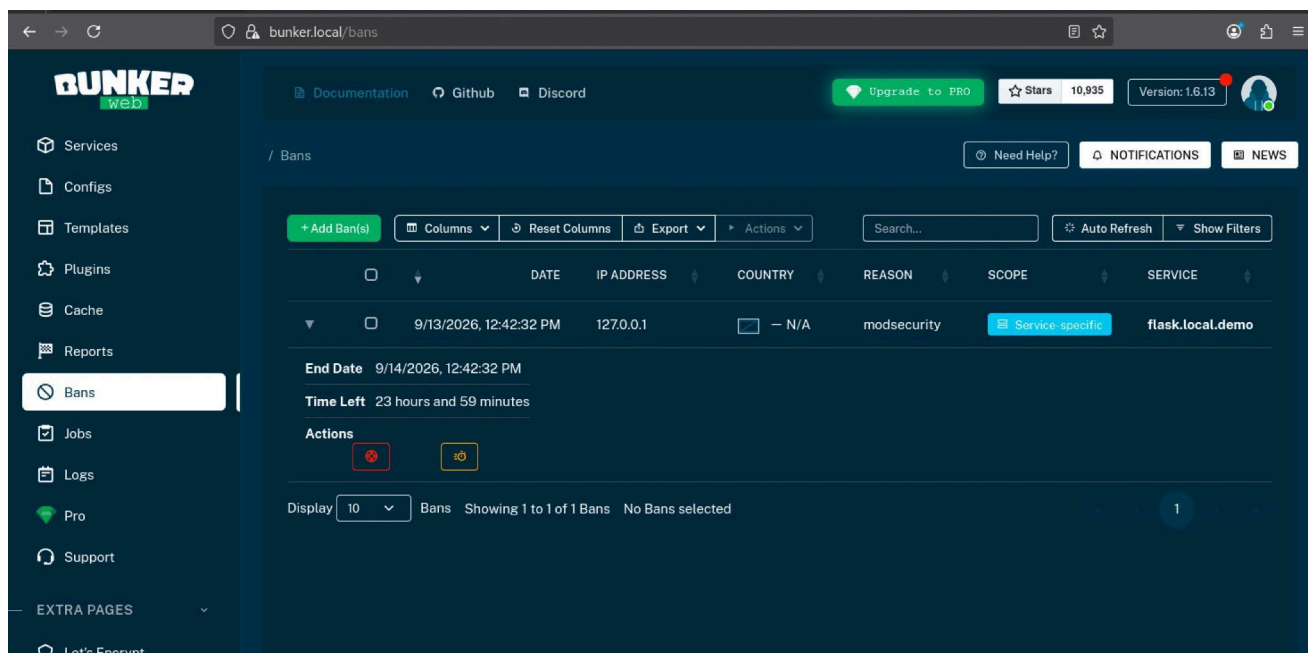


Figure 8. Evidence captured during controlled BunkerWeb WAF testing.

Screenshot 9- Final Protected-Service 403 Response

A further request to the protected service is rejected with BunkerWeb's 403 page. The client is marked Forbidden while BunkerWeb and the web server remain Working. Together with the Reports and Bans screenshots, this provides end-to-end evidence of WAF enforcement.

Key evidence: The protected service continues to enforce the WAF decision and returns 403.

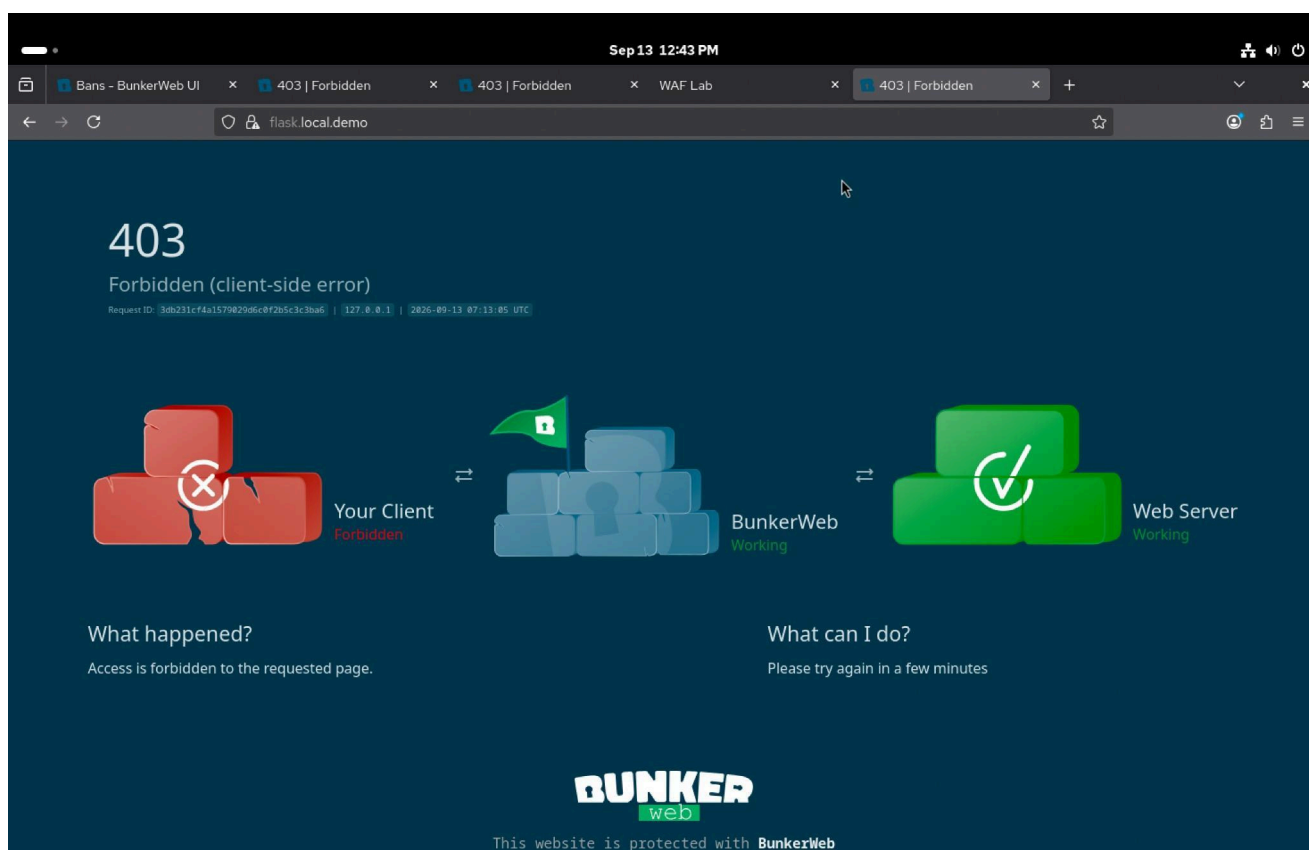


Figure 9. Evidence captured during controlled BunkerWeb WAF testing.

4. Technical Interpretation

The evidence demonstrates a layered security response rather than a single point-in-time block. The attack requests first appear as HTTP input at the application boundary. BunkerWeb inspects the request using its ModSecurity security controls. When a request matches an attack signature or detection rule, the WAF can reject it before the Flask application processes it as a normal request.

4.1 SQL Injection

SQL Injection occurs when untrusted input can influence the structure of a database query. In this validation, the login form is used to submit a SQL-injection test value. The BunkerWeb report records Rule 942100 and identifies the username argument as the source of the detection. The resulting 403 response confirms that the request was denied.

4.2 Cross-Site Scripting (XSS)

XSS involves injecting browser-interpreted script content through an application input. The controlled script-tag test triggered XSS detections including Rules 941100 and 941110. The report therefore provides both the input location and the security-rule classification, while the 403 response confirms enforcement.

4.3 Path Traversal

Path Traversal attempts to move outside an application's intended directory structure using traversal sequences. The report identifies Rule 930100 and shows the suspicious request URI and file argument. The browser subsequently receives 403 Forbidden, demonstrating that the WAF rejected the request.

4.4 Ban Enforcement

The Bans screenshot shows an enforcement entry with the reason 'modsecurity' and a service-specific scope for the protected Flask service. This demonstrates that the security workflow can extend beyond an individual blocked request to a temporary ban decision.

5. Security Recommendations

WAF protection should be treated as a defense-in-depth control rather than a replacement for secure application development. The Flask application should still validate input, use parameterized database queries, apply context-aware output encoding, restrict file-access paths, and implement secure authentication controls.

Logging and alerting should also be retained so that repeated attack activity can be investigated.

6. Conclusion

The completed validation demonstrates that the BunkerWeb deployment can detect and block representative web attacks and provide operational evidence through its Reports and Bans interfaces. The screenshots collectively verify SQL Injection detection, XSS detection, Path Traversal detection, 403 enforcement, security reporting, and service-specific ban behavior.

Final outcome: The Flask application is protected by a functioning BunkerWeb WAF security layer, with visible evidence of attack detection and enforcement.